



External JSON-2 API

New in version 19.0.

Odoo is usually extended internally via modules, but many of its features and all of its data are also available externally for analysis or integration with various other softwares. Part of the **Models** API is easily available over HTTP via the `/json/2` endpoint.

Tip

The actual models, fields and methods available are specific to every database and can be consulted on their `/doc` page.

Note

Access to data via the external API is only available on *Custom* Odoo pricing plans. Access to the external API is not available on *One App Free* or *Standard* plans. For more information visit the [Odoo pricing page](#) or reach out to your Customer Success Manager.

API

Request

Post a JSON object at the `/json/2/<model>/<method>` URL.

HTTP Headers

Host:

Required, the hostname of the server.

Authorization:

Required, `bearer` followed by an [API key](#).

Content-Type:

Required, `application/json`, a charset is recommended.

X-Odoo-Database:

Optional, the name of the database to connect to.

User-Agent:

Recommended, the name of your software.

Required, the technical model name.

method:

Required, the method to execute.

Body JSON object

ids:

An array of record ids on which to execute the method. Empty or omitted when calling an `@api.model` - decorated method.

context:

Optional, an object of additional values. e.g. `{"lang": "en_US"}` .

param:

As many time as needed, the method's parameters.

Example

```
POST /json/2/res.partner/search_read HTTP/1.1
Host: mycompany.example.com
X-Odoo-Database: mycompany
Authorization: bearer 6578616d706c65206a736f6e20617069206b6579
Content-Type: application/json; charset=utf-8
User-Agent: mysoftware python-requests/2.25.1
```

```
{
  "context": {
    "lang": "en_US"
  },
  "domain": [
    ["name", "ilike", "%deco%"],
    ["is_company", "=", true]
  ],
  "fields": ["name"]
}
```

Response

In case of **success**, a **200** status with the JSON-serialized return value of the called method in the body.

```
Content-Type: application/json; charset=utf-8
```

```
[
  {"id": 25, "name": "Deco Addict"}
]
```

In case of **error**, a **4xx/5xx** status with a JSON-serialized error object in the body.

name:

The fully qualified name of the Python exception that occurred.

message:

The exception message, usually the same as `arguments[0]`.

arguments:

All the exception arguments.

context:

The context used by the request.

debug:

The exception traceback, for debugging purpose.

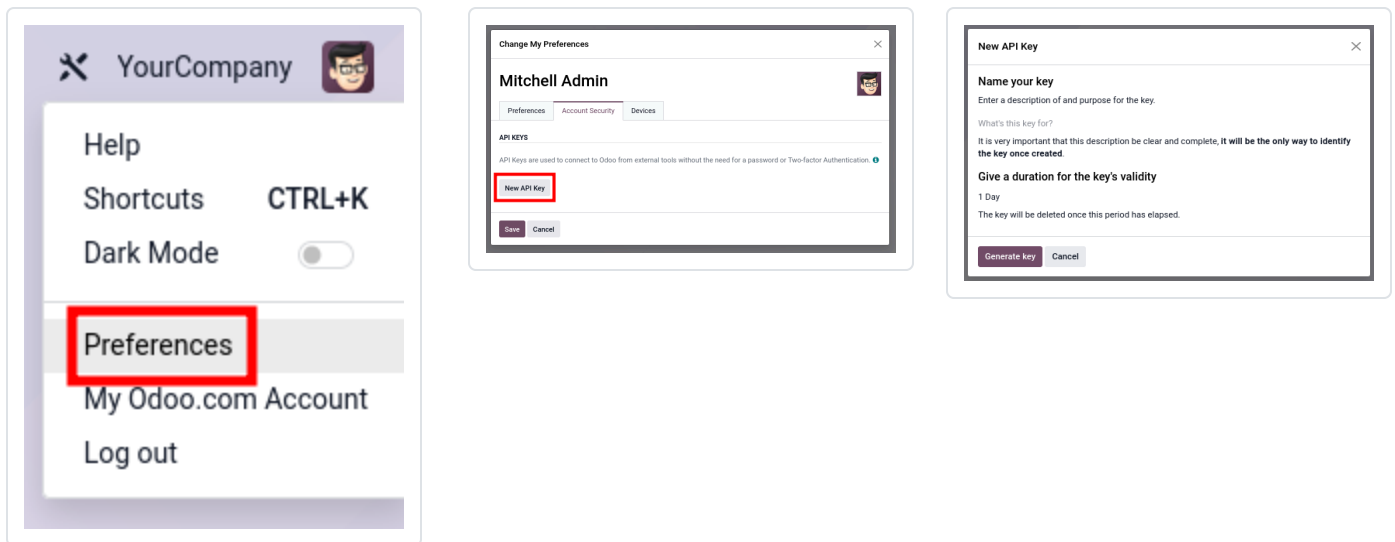
Example

HTTP	Debug
<pre> HTTP/1.1 401 Unauthorized Content-Type: application/json; charset=utf-8 { "name": "werkzeug.exceptions.Unauthorized", "message": "Invalid apikey", "arguments": ["Invalid apikey", 401], "context": {}, "debug": "Traceback (most recent call last):\n File \"/opt/Od </pre>	



Configuration

Create a new API key for a user via **Preferences** > **Account Security** > **New API Key**.



Both a description and a duration are needed to create a new API key. The description makes it possible to identify the key, and to determine later whether the key is still in use or should be removed. The duration determines the lifetime of the key, after which the key becomes invalid. It is recommended to set a short duration (typically one day) for interactive usage. For security reasons, it is not possible to create keys that last for more than three months. This means that long lasting keys must be rotated at least once every three months.

The **Generate Key** button creates a strong 160-bits random key. The key value is displayed only once during creation and cannot be retrieved later. Copy the key immediately and store it securely. If the key is compromised or lost, delete it immediately and generate a new one.

Please refer to [OWASP's Secrets Management Cheat Sheet](#) for further guidance on the management of API keys.

Access Rights

The JSON-2 API uses the standard [security models of Odoo](#). All operations are validated against the access rights, record rules and field accesses of the user.

For **interactive usage**, such as discovering the API or running one-time scripts, it is fine to use a **personal account**.

For **extended automated usage**, such as an integration with another software, it is recommended to create and use **dedicated bot users**. Using dedicated bot users has several

- The password can be set empty to disable login/password authentication, limiting the likelihood of the account getting compromised.
- The **Access Log fields** use the bot account. No user is impersonalized.

Database

Depending on the deployment, the `Host` and/or `X-Odoo-Database` request headers might be required. The `Host` header is required by HTTP/1.1 and is needed on servers where Odoo is installed next to other web applications, so that a web-server/reverse-proxy is able to route the request to the Odoo server. The `X-Odoo-Database` header is required when a single Odoo server hosts multiple databases and the **dbfilter** wasn't configured to use the `Host` header.

Most HTTP client libraries automatically set the `Host` header using the connection URL.

Transaction

All calls to the JSON-2 endpoint run in their own SQL transaction. The transaction is committed in case of success and is discarded in case of error. Using the JSON-2 API, it is not possible to chain multiple calls inside a single transaction. It means that one must be cautious when making multiple consecutive calls, as the database might be modified by other concurrent transactions. This is especially dangerous when performing operations related to reservations, payments, and such.

The solution is to always call a single method that performs all the related operations in a single transaction. This way, the data is guaranteed to stay consistent: either everything is done (success, commit), or nothing is done (error, rollback).

In the ORM, the `search_read` method is an example of a single method that performs multiple operations (`search` then `read`) in a single transaction. If a concurrent request removes one of the records `search` retrieves, then there is a risk that subsequent calls to `read` fail for a missing record error. Such a problem cannot occur in `search_read` , as the system guarantees proper isolation between transactions.

In business models, those methods are often prefixed by `action_` , such as `sale.order` 's `action_confirm` method, which verifies that a sales order is valid before confirming it.

When no method exists for a set of related operations, a new one can be created in a dedicated module.

Code Example

The following examples showcase how to execute two of the **common ORM methods** on a dummy database `mycompany` hosted on the dummy website `https://mycompany.example.com`. Its **dynamic documentation** would be available at <https://mycompany.example.com/doc>.

Python

JavaScript

Bash

```
(async () => {
  const BASE_URL = "https://mycompany.example.com/json/2";
  const API_KEY = ; // get it from a secure location
  const headers = {
    "Content-Type": "application/json",
    "Authorization": "bearer " + API_KEY,
    "X-Odoo-Database": DATABASE,
  }

  const reqSearch = {
    method: "POST",
    headers: headers,
    body: {
      context: {lang: "en_US"},
      domain: [
        ["name", "ilike", "%deco%"],
        ["is_company", "=", true],
      ],
    },
  };

  const resSearch = await fetch(BASE_URL + "/res.partner/search");
  if (!response.ok) throw new Error(resSearch.json());
  const ids = await resSearch.json();

  const reqRead = {
    method: "POST",
    headers: headers,
    body: {
      ids: ids,
      context: {lang: "en_US"},
    },
  };
});
```

```
if (!response.ok) throw new Error(resRead.json());
const names = await resRead.json();
console.log(names);
})();
```

The above example is equivalent to running:

```
Model = self.env["res.partner"].with_context({"lang": "en_US"})
records = Model.search([("name", "ilike", "%deco%"), ("is_company", "=", True)])
return json.dumps(records.ids)
```

Then, in a new transaction:

```
records = self.env["res.partner"].with_context({"lang": "en_US"}).browse(ids)
names = records.read(["name"])
return json.dumps(names)
```

Dynamic Documentation

Under construction

Migrating from XML-RPC / JSON-RPC

Both the XML-RPC and JSON-RPC APIs at endpoints `/xmlrpc`, `/xmlrpc/2` and `/jsonrpc` are scheduled for removal in Odoo 20 (fall 2026). Both RPC APIs expose the three same services: `common`, `db` (database) and `object`. All three services are deprecated.

Note

The other controllers `@route(type='jsonrpc')` (known until Odoo 18 as `type='json'`) are not subject to this deprecation notice.

Common service

```
3. authenticate(db, login, password, user_agent_env)
```

The version function is replaced by the `/web/version` endpoint.

```
GET /web/version HTTP/1.1
```

```
HTTP/1.1 200 OK
```

```
Content-Type: application/json
```

```
{"version_info": [19, 0, 0, "final", 0, ""], "version": "19.0"}
```

The two `login` and `authenticate` functions return the user ID corresponding to the user after a successful login. The user ID and password are necessary for subsequent RPC calls to the *object* service. The JSON-2 API uses a different authentication scheme where neither the user ID nor the password are used. It is still possible to retrieve the user's own ID by sending a JSON-2 request to `res.users/context_get` with no ID (the current user is extracted from the API key).

Database service

See also

Database Manager Security

The db service defines 13 fonctions:

- ```
create_database(master_pwd, db_name, demo, lang, user_password, login, country_code,
1. phone)
2. duplicate_database(master_pwd, db_original_name, db_name, neutralize_database)
3. drop(master_pwd, db_name)
4. dump(master_pwd, db_name, format)
5. restore(master_pwd, db_name, data, copy)
6. change_admin_password(master_pwd, new_password)
7. rename(master_pwd, old_name, new_name)
8. migrate_databases(master_pwd, databases)
9. db_exist(db_name)
```



### 13. `server_version()`

Many of those function are accessible via the `/web/database` controllers. Those controllers work hand-in-hand with the HTML form at `/web/database/manager` and are accessible via HTTP.

The following controllers use the verb `POST` and content-type `application/x-www-form-urlencoded`.

1. `/web/database/create` takes inputs `master_pwd`, `name`, `login`, `password`, `demo`, `lang`, and `phone`.
2. `/web/database/duplicate` takes inputs `master_pwd`, `name`, `new_name`, and `neutralize_database` (not neutralized by default).
3. `/web/database/drop` takes inputs `master_pwd` and `name`.
4. `/web/database/backup` takes inputs `master_pwd`, `name`, and `backup_format` (zip by default), and returns the backup in the http response.
5. `/web/database/change_password` takes inputs `master_pwd` and `master_pwd_new`.

The following controller uses the verb `POST` and content-type `multipart/form-data`.

- `/web/database/restore` takes inputs `master_pwd`, `name`, `copy` (not copied by default) and `neutralize` (not neutralized by default), it takes a file input `backup_file`.

The following controller uses the verb `POST` and content-type `application/json-rpc`.

- `/web/database/list` takes an empty JSON object as input, and returns the database list under the JSON response's `result` entry.

The remaining function are: `server_version`, which exists under `/web/version`, `list_lang`, and `list_countries`, which exist via JSON-2 on the `res.lang` and `res.country` models, and `migrate_databases`, which as non-programmable API at the moment.

## Object service

The object service defines 2 fonctions:

1. `execute(db, uid, passwd, model, method, *args)`
2. `execute_kw(db, uid, passwd, model, method, args, kw={})`

They both give for access to all public model methods, including the generic ORM ones.

takes an `args` list of positional arguments and an optional `kw` dict of keyword arguments.

The records IDs are extracted from the first `args`. When the called method is decorated with `@api.model`, no record ID is extracted, and `args` is left as-is. It is only possible to give a context with `execute_kw`, as it is extracted from the keyword argument named `context`.

## Example

To run the following:

```
(env['res.partner']
 .with_user(2) # admin
 .with_context(lang='en_US')
 .browse([1, 2, 3])
 .read(['name'], load=None)
)
```

Using XML-RPC (JSON-RPC would be similar):

```
from xmlrpc.client import ServerProxy
object = ServerProxy(...)
ids = [1, 2, 3]
fields = ['name']
load = None

object.execute("database", 2, "admin", "res.partner", "read", ids, fields, load)
object.execute("database", 2, "admin", "res.partner", "search", [
 ids,
 fields,
], {
 "context": {"lang": "en_US"},
 "load": load,
})
```

The JSON-2 API replaces the object service with a few differences. The database must only be provided (via the `x-Odoo-Database` HTTP header) on systems where there are multiple databases available for a same domain. The login/password authentication scheme is replaced by an API key (via the `Authorization: bearer` HTTP header). The `model` and `method` are placed in the URL. The request body is a JSON object with all the methods arguments, plus

## Example

Using JSON-2:

```
import requests

DATABASE = ...
DOMAIN = ...
API_KEY = "6578616d706c65206a736f6e20617069206b6579"

requests.post(
 f"https://{DOMAIN}/json/2/res.partner/read",
 headers={
 # "X-Odoo-Database": DATABASE, # only when DOMAIN isn't enough
 "Authorization": f"bearer {API_KEY}",
 },
 json={
 "ids": [1, 2, 3],
 "context": {"lang": "en_US"},
 "fields": ["name"],
 "load": None,
 },
).json()
```

## Get Help

[Contact Support](#)[Ask the Odoo Community](#)